

Backpropagation: A Line-by-Line Beginner's Guide

Understanding the "Smart Number" Logic

1 The Core Idea: Sensitivity

In Deep Learning, we don't just care about the answer. We care about **sensitivity**. If we have a result L , we want to know: *"If I change input a by a tiny bit, how much does L change?"* This sensitivity is called the **Gradient** (grad).

2 Step 1: The "Smart Box" (Value class)

Every number in our program is wrapped in this class.

```
class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data          # The actual value (e.g., 2.0)
        self.grad = 0.0          # The Sensitivity (Starts at 0)
        self._prev = set(_children) # The "Parents" of this number
        self._backward = lambda: None # The recipe to calculate gradients
```

3 Step 2: Defining the Example

Let's use the exact numbers from the notebook:

$$L = (a \times b + c) \times f$$

```
a = Value(2.0)
b = Value(-3.0)
c = Value(10.0)
f = Value(-2.0)

e = a * b      # Intermediate step
d = e + c      # Intermediate step
L = d * f      # Final result
```

4 Step 3: Line-by-Line Walkthrough

4.1 The Forward Pass (Left to Right)

This is just normal math, but the computer is building a "Family Tree" in the background.

1. $e = a \times b$: $2.0 \times -3.0 = -6.0$. Box e is created. It remembers its parents are a and b .
2. $d = e + c$: $-6.0 + 10.0 = 4.0$. Box d is created. It remembers its parents are e and c .
3. $L = d \times f$: $4.0 \times -2.0 = -8.0$. Box L is the final result.

4.2 The Backward Pass (Right to Left)

This is where the magic happens. We calculate `grad` (influence).

The Goal

We start at the end and set `L.grad = 1.0`. Then we work backward.

1. Influence of d and f on L Since $L = d \times f$, if you increase d , L changes by f .

- `d.grad = f.data * L.grad` $\rightarrow -2.0 \times 1.0 = -2.0$
- `f.grad = d.data * L.grad` $\rightarrow 4.0 \times 1.0 = 4.0$

2. Influence of e and c on d Since $d = e + c$, addition just "passes" the influence back.

- `e.grad = 1.0 * d.grad` $\rightarrow 1.0 \times -2.0 = -2.0$
- `c.grad = 1.0 * d.grad` $\rightarrow 1.0 \times -2.0 = -2.0$

3. Influence of a and b on e Since $e = a \times b$, we use the multiplication rule again.

- `a.grad = b.data * e.grad` $\rightarrow -3.0 \times -2.0 = 6.0$
- `b.grad = a.data * e.grad` $\rightarrow 2.0 \times -2.0 = -4.0$

5 Conclusion

At the end, `a.grad` is **6.0**. This means if you increase `a` by 0.001, the final result `L` will increase by 0.006. The computer has successfully calculated the "Chain Rule" without us writing any calculus!